```
import re
import nltk
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
text_col = 'tweet'
label_col = 'label'
```

```
train_df = pd.read_csv('/content/train.csv')
test_df = pd.read_csv('/content/test.csv')
```

```
train_df.dropna(subset=[text_col, label_col], inplace=True)
test_df.dropna(subset=[text_col], inplace=True)
```

```
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
```

```
def clean_text(text):
    text = re.sub(r"http\S+|www\S+|https\S+", '', str(text))  # Remove URLs
    text = re.sub(r'@\w+|#\w+', '', text)  # Remove Twitter handles and hashtags
    text = re.sub(r'[^A-Za-z\s]', '', text)  # Remove punctuation and non-alphabetic chars
def clean_text(text):
    text = re.sub(r"http\S+|www\S+|https\S+", '', str(text))  # Remove URLs
    text = re.sub(r'@\w+|#\w+', '', text)  # Remove Twitter handles and hashtags
    text = re.sub(r'[^A-Za-z\s]', '', text)  # Remove punctuation and non-alphabetic chars
```

```
    text = text.lower()  # Lowercase text
    return ' '.join([lemmatizer.lemmatize(word) for word in text.split() if word not in stop
```

```
train_df['clean_text'] = train_df[text_col].apply(clean_text)
test_df['clean_text'] = test_df[text_col].apply(clean_text)
```

```
vectorizer = TfidfVectorizer(max_features=2000)
X_train = vectorizer.fit_transform(train_df['clean_text']).toarray()
y_train = train_df[label_col]
X_test = vectorizer.transform(test_df['clean_text']).toarray()
```

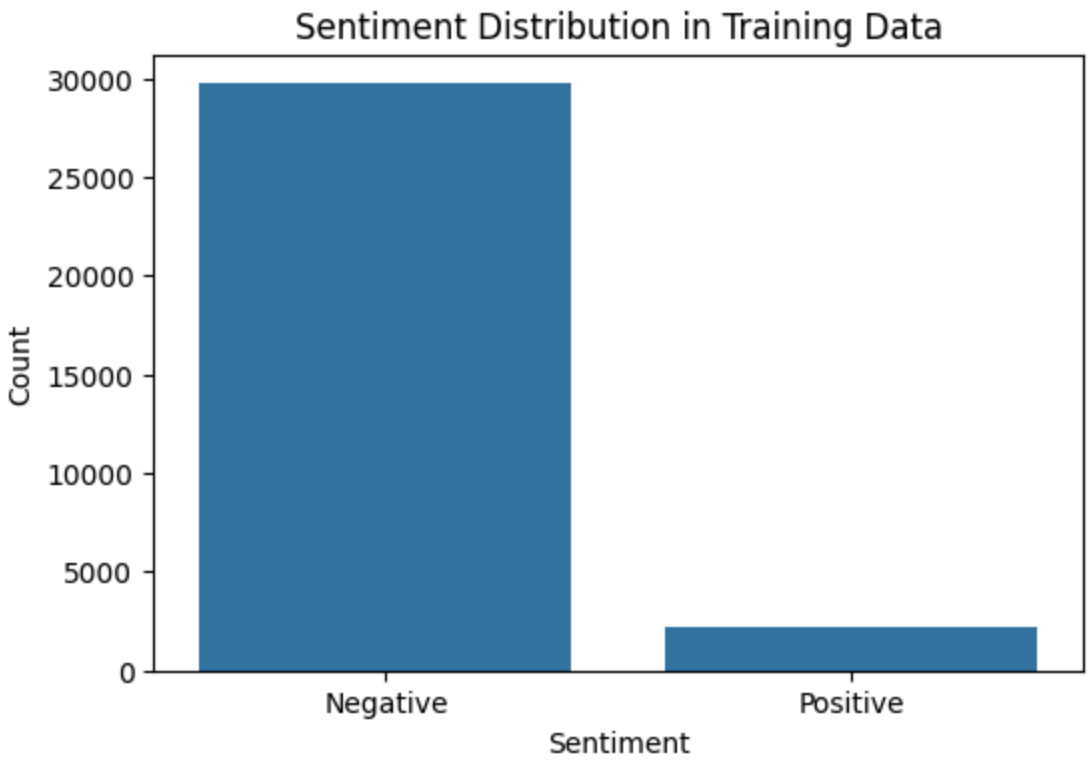
```
clf = LogisticRegression()
clf.fit(X_train, y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

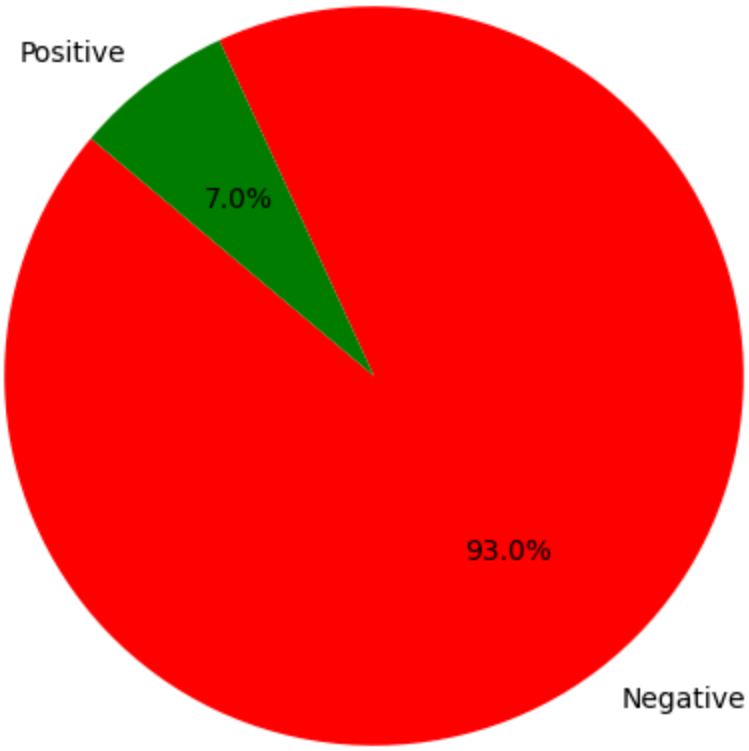
```
y_train_pred = clf.predict(X_train)
print("Training Accuracy:", accuracy_score(y_train, y_train_pred))
```

```
Training Accuracy: 0.9469369876728615
```

```
plt.figure(figsize=(6, 4))
sns.countplot(x=label_col, data=train_df)
plt.title("Sentiment Distribution in Training Data")
plt.xticks([0, 1], ['Negative', 'Positive'])
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.show()
```




```
plt.figure(figsize=(6, 6))
train_df[label_col].value_counts().plot.pie(
    autopct='%1.1f%%', labels=['Negative', 'Positive'], colors=['red', 'green'], startangle=
plt.title("Sentiment Ratio in Training Data")
plt.ylabel("")
plt.show()
plt.title("Sentiment Ratio in Training Data")
plt.ylabel("")
plt.show()
```

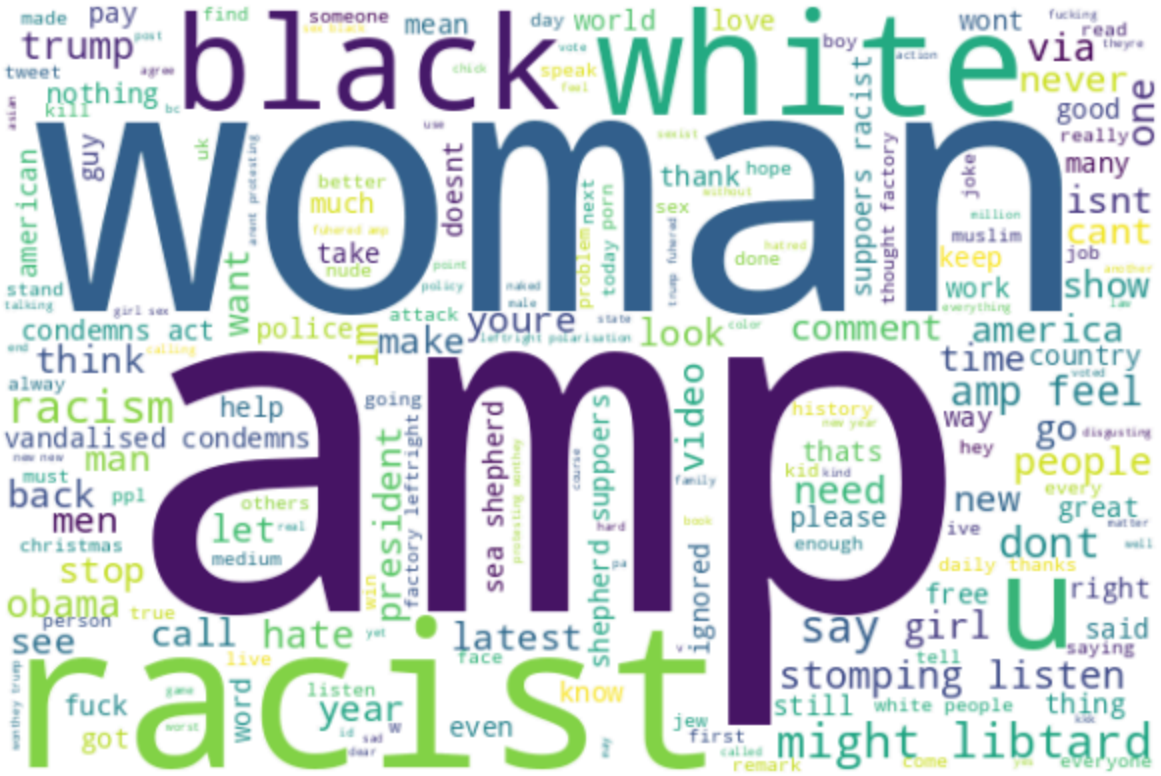

Sentiment Ratio in Training Data
Positive
7.0%
93.0%
Negative


Sentiment Ratio in Training Data
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

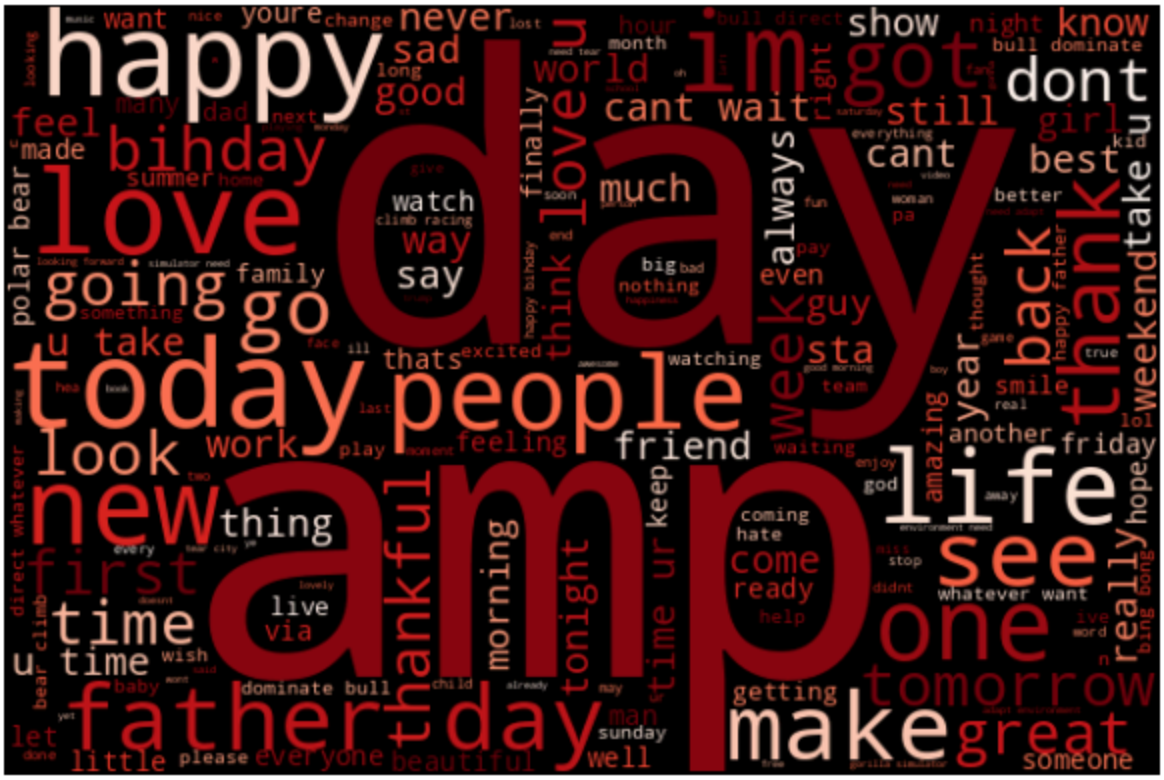
```
positive_text = " ".join(train_df[train_df[label_col] == 1]['clean_text'])
wordcloud_pos = WordCloud(width=600, height=400, background_color='white').generate(positive
plt.figure(figsize=(8, 5))
plt.imshow(wordcloud_pos, interpolation='bilinear')
plt.axis("off")
plt.title("WordCloud - Positive Tweets")
plt.show()

negative_text = " ".join(train_df[train_df[label_col] == 0]['clean_text'])
wordcloud_neg = WordCloud(width=600, height=400, background_color='black', colormap='Reds').
plt.figure(figsize=(8, 5))
plt.imshow(wordcloud_neg, interpolation='bilinear')
plt.axis("off")
plt.title("WordCloud - Negative Tweets")
plt.show()
```

WordCloud - Positive Tweets



WordCloud - Negative Tweets




```
if 'label' in test_df.columns:
    y_test = test_df['label']
    y_test_pred = clf.predict(X_test)
    print("Test Accuracy:", accuracy_score(y_test, y_test_pred))
```

```
    print("Classification Report:\n", classification_report(y_test, y_test_pred))

    cm = confusion_matrix(y_test, y_test_pred)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Negative', 'Positive'],
    plt.title("Confusion Matrix - Test Set")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()
else:
    test_df['predicted_sentiment'] = clf.predict(X_test)
    print(test_df[[text_col, 'predicted_sentiment']].head())
```

```
                                               tweet  predicted_sentiment
0  #studiolife #aislife #requires #passion #dedic...                    0
1   @user #white #supremacists want everyone to s...                    0
2  safe ways to heal your #acne!!    #altwaystohe...                    0
3  is the hp and the cursed child book up for res...                    0
4    3rd #bihday to my amazing, hilarious #nephew...                    0
```